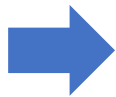


学习内容

CONTENTS



1

绪论

2

Python基础

3

机器学习及其应用

4

人工神经网络与深度学习

5

大模型与AI智能体

6

知识表示与知识图谱

7

不确定性推理方法

8

智能计算及其应用

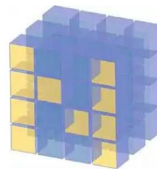


pandas

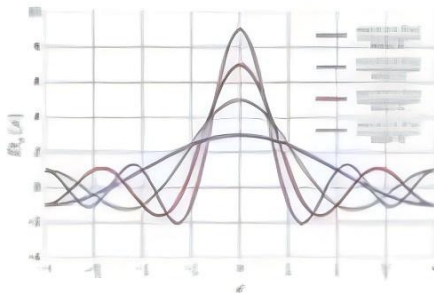
$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$



matplotlib



NumPy



2.1 基础知识（字符串、列表、元组、字典等数据类型，选择和循环等流程控制，函数定义及调用，面向对象设计

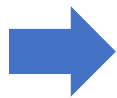
2.2 AI库及使用

- 熟练运用，实现流程控制，掌握函数定义与调用逻辑，理解面向对象设计的类、对象、继承等概念
- 重点：字符串、列表、元组、字典的数据操作方法，函数定义与调用，NumPy 的数组运算、Pandas 的数据清洗与分析、Matplotlib 的数据可视化



AI库及使用

学习内容 CONTENTS



1

Numpy

2

Pandas

3

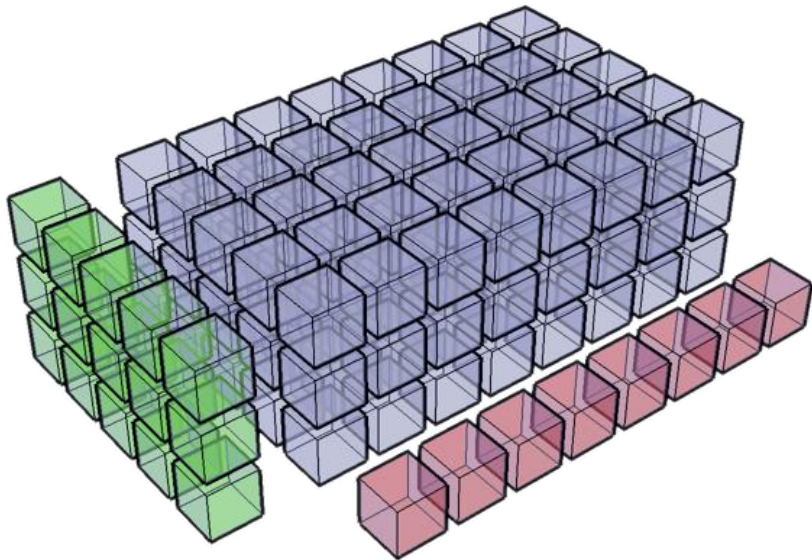
Matplotlib

4

Scikit-learn

- **Numpy**(Numeric Python)是一个开源的科学计算的扩展库

- 创建n维数组
- 对数组进行函数运算
- 数值积分
- 线性代数运算
- 傅里叶变换
- 随机数产生
-



标准的Python中用list（列表）保存值，可以当做数组使用，但因为列表中的元素可以是任何对象，所以浪费了CPU运算时间和内存。

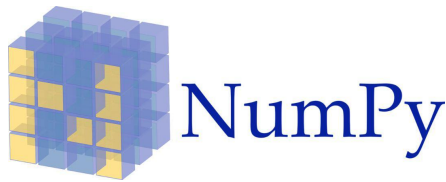
NumPy诞生为了弥补这些缺陷。它提供了两种基本的对象：

ndarray: 全称（n-dimensional array object）是储存单一数据类型（如int64、float64）的多维数组。

ufunc: 全称（universal function object）它是一种能够对数组进行处理的函数。

NumPy的官方文档：

<https://numpy.org/doc/stable/>



N 维数组对象ndarray: 用于存放同类型元素的多维数组,以 0 下标为开始进行集合中元素的索引。

```
import numpy as np          # 导入NumPy工具包
# 创建 ndarray
score = np.array([[80, 89, 86, 67, 79],[78, 97, 89, 67, 81]])
# 打印结果
score
```

```
array([[80, 89, 86, 67, 79],
       [78, 97, 89, 67, 81]])
```

```
print(score.shape, score.ndim, score.size, score.dtype)
(2, 5) 2 10 int64
```

语文	数学	英语	政治	体育
80	89	86	67	79
78	97	89	76	81

- ndarray.shape 数组维度的元组

- ndarray.ndim 数组维数

- ndarray.size 数组中的元素数量

```
np.array([1, 2, 3, 4], float) #将列表 [1, 2, 3, 4] 转换为浮点数类型的NumPy数组。数组元素的类型
array([1., 2., 3., 4.])
```

N 维数组对象ndarray: 用于存放同类型元素的多维数组,以 0 下标为开始进行集合中元素的索引。

```
import numpy as np          # 导入NumPy工具包
data = np.arange(12).reshape(3, 4)  # 创建一个3行4列的数组
print(type(data))
print(data)
```

```
<class 'numpy.ndarray'>
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

ndarray对维数没有限制，维度称为轴（axis），
[] 从内到外分别为第0轴，第1轴，...

```
print(data.ndim) # 数组维度的个数，输出结果2，表示二维数组
print(data.shape , data.size , data.size )
```

```
2
```

```
(3, 4) 12 12
```

```
np.array([7,2,9,10])
```

#创建一维数组

```
np.array([[5.2,3.0,4.5],[9.1,0.1,0.3]])
```

#创建二维数组

```
np.random.randint(0, 10, size=(4, 3, 2))
```

#创建三维数组

3D array

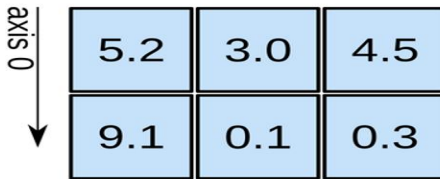
1D array



axis 0

shape(4,)

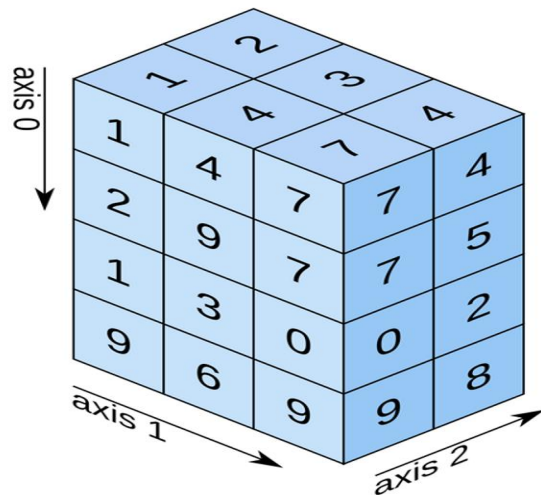
2D array



axis 0

axis 1

shape(2,3)



axis 0

axis 1

axis 2

shape(4,3,2)

`np.zeros((3, 4))` #创建一个全0数组

```
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])
```

`np.ones((3, 4), dtype= 'float64')`

```
array([[1., 1., 1., 1.],
       [1., 1., 1., 1.],
       [1., 1., 1., 1.]])
```

`np.eye(3)` #创建方阵对角矩阵

```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

名称	描述
<code>np.bool</code>	用一个字节存储的布尔类型 (True或False)
<code>np.int8</code>	一个字节大小, -128 至 127
<code>np.int16</code>	整数, -32768 至 32767
<code>np.int32</code>	整数, -231 至 232-1
<code>np.int64</code>	整数, -263 至 263-1
<code>np.uint8</code>	无符号整数, 0 至 255
<code>np.uint16</code>	无符号整数, 0 至 65535
<code>np.uint32</code>	无符号整数, 0 至 232-1
<code>np.uint64</code>	无符号整数, 0 至 264-1
<code>np.float16</code>	半精度浮点数: 16位, 正负号1位, 指数5位, 精度10位
<code>np.float32</code>	单精度浮点数: 32位, 正负号1位, 指数8位, 精度23位
<code>np.float64</code>	双精度浮点数: 64位, 正负号1位, 指数11位, 精度52位
<code>np.complex64</code>	复数, 分别用两个32位浮点数表示实部和虚部
<code>np.complex128</code>	复数, 分别用两个64位浮点数表示实部和虚部
<code>np.object_</code>	python对象
<code>np.string_</code>	字符串
<code>np.unicode_</code> <code>unicode</code>	类型

NumPy提供了专门用于生成ndarray的函数，提高创建ndarray的速度。

`numpy.arange([start, ]stop, [step, ]dtype=None, *, device=None, like=None)`

生成一个在 `[start, stop)` 区间内，以 `step` 为步长的等差数列。返回的是 `ndarray` 参数：

`start`: 开始位置，数字，可选项，默认起始值为0

`stop`: 停止位置，数字

`step`: 步长，数字，可选项，默认步长为1，如果指定了`step`，则还必须给出`start`。

`dtype`: 输出数组的类型。如果未给出`dtype`，则从其他输入参数推断数据类型。

```
# 生成等间隔的数组
```

```
b = np.arange(0, 90, 10) #生成一个从0开始，步长为10，且小于90的等差数列  
b
```

```
array([ 0, 10, 20, 30, 40, 50, 60, 70, 80])
```

`numpy.linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None, axis=0)`

在指定的区间内生成等间隔的数值。

参数：

start: 序列的起始值。

stop: 序列的终止值。

num: 要生成的等间距样本数量，默认为50。

endpoint: 如果为True，则包含stop值；如果为False，则不包含终止值。

retstep: 如果为True，则返回样本和步长；否则只返回样本。

dtype: 数组的类型。

axis: 沿着哪个轴生成数值。

```
# 生成等间隔的数组
```

```
a = np.linspace(0, 90, 10) #在区间[0, 90]上生成10个等间距的数值，返回一维数组  
a
```

```
array([ 0., 10., 20., 30., 40., 50., 60., 70., 80., 90.])
```

```
from numpy import array
a = array([[ 0, 1, 2, 3, 4, 5],
           [10,11,12,13,14,15],
           [20,21,22,23,24,25],
           [30,31,32,33,34,35]])
```

a.shape

(4, 6)

```
# 在转换形状的时候，一定要注意数组的元素匹配
# 只是将形状进行了修改，但并没有将行列进行转换
b = a.reshape((3, 8))
b
```

```
array([[ 0, 1, 2, 3, 4, 5, 10, 11],
       [12, 13, 14, 15, 20, 21, 22, 23],
       [24, 25, 30, 31, 32, 33, 34, 35]])
```

```
# 数组的形状被修改为: (2, 12), -1: 表示同过自动计算得到此处的值
c = a.reshape((-1, 12))
c
```

```
array([[ 0, 1, 2, 3, 4, 5, 10, 11, 12, 13, 14, 15],
       [20, 21, 22, 23, 24, 25, 30, 31, 32, 33, 34, 35]])
```

```
# 调用flatten()方法：将二维数组展平为一维数组
a.flatten()
```

```
array([ 0, 1, 2, 3, 4, 5, 10, 11, 12, 13, 14, 15, 20, 21, 22, 23,
       24, 25, 30, 31, 32, 33, 34, 35])
```

类型修改:

```
aarr = np.array([[[1, 2, 3], [4, 5, 6]], [[12, 3, 34], [5, 6, 7]]])  
arr.dtype
```

```
dtype('int64')
```

```
arr.astype(np.float32)
```

```
array([[[ 1., 2., 3.],  
        [ 4., 5., 6.]],  
       [[12., 3., 34.],  
        [ 5., 6., 7.]]], dtype=float32)
```

ndarray的切片和list是一样的。

```
> a = np.arange(10)
```

```
> a
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
a[5]
```

```
a[3:5]
```

```
a[:5]
```

```
a[:-1]
```

```
5
```

```
[3, 4]
```

```
[0, 1, 2, 3, 4]
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8]
```

```
a[1:-1:2]
```

```
a[::-1]
```

```
a[5:1:-2]
```

```
[1, 3, 5, 7]
```

```
[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

```
[5, 3]
```

可以通过切片的对ndarray中的元素进行更改。

```
> a[2:4] = 100, 101
```

```
> a
```

```
array([ 0,  1, 100, 101,  4,  5,  6,  7,  8,  9])
```

ndarray通过切片产生一个新的数组b，b和a共享同一块数据存储空间。

```
> b = a[3:7]
> b[2] = -10
```

b

a

```
[101, 4, -10, 6] [ 0, 1, 100, 101, 4, -10, 6, 7, 8, 9]
```

如果想改变这种情况，我们可以用列表对数组元素切片。

```
> b = a[[3, 3, -3, 8]]
> b
array([3, 3, 7, 8])
```

```
> b[2] = 100
```

b

a

```
[3, 3, 100, 8] [ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

NumPy的多维数组和一维数组类似。多维数组有多个轴。
 我们前面已经提到从内到外分别是第0轴，第1轴…

```
a = np.arange(0, 60, 10).reshape(-1, 1) + np.arange(0, 6)
```

```
array([[ 0,  1,  2,  3,  4,  5],
       [10, 11, 12, 13, 14, 15],
       [20, 21, 22, 23, 24, 25],
       [30, 31, 32, 33, 34, 35],
       [40, 41, 42, 43, 44, 45],
       [50, 51, 52, 53, 54, 55]])
```

`a[0, 3:5]`

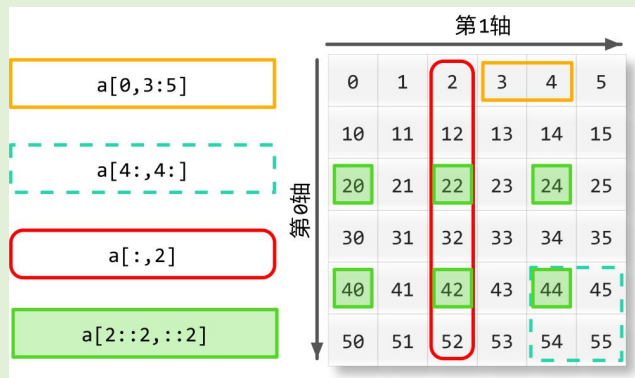
`a[4:, 4:]`

`a[2::2, ::2]`

`[3, 4]`

`[[44, 45],
[54, 55]]`

`[[20, 22, 24],
[40, 42, 44]]`



#上面方法对于数组的切片都是共享原数组的储存空间的。

如果我们想创立原数组的副本，我们可以用整数元组，列表，整数数组，布尔数组进行切片。

```
>>> a[0,3:5]
array([3,4])
>>> a[4:,4:]
array([[44,45],[54,55]])
>>> a[:,2]
array([2,12,22,32,42,52])
>>> a[2::2,::2]
array([[20,22,24],
       [40,42,44]])
```

0	1	2	3	4	5
10	11	12	13	14	15
20	21	22	23	24	25
30	31	32	33	34	35
40	41	42	43	44	45
50	51	52	53	54	55

```
>>> a[(0,1,2,3,4),(1,2,3,4,5)]
array([1,12,23,34,45])
>>> a[3:,[0,2,5]]
array([[30,32,35],
       [40,42,45],
       [50,52,55]])
>>> mask=np.array([1,0,1,0,0,1],
                   dtype=np.bool)
>>> a[mask,2]
array([2,22,52])
```

第 0 轴

0	1	2	3	4	5
10	11	12	13	14	15
20	21	22	23	24	25
30	31	32	33	34	35
40	41	42	43	44	45
50	51	52	53	54	55

第 1 轴

ufunc是**universal function**的简称，它是一种能对**数组每个元素进行运算**的函数。N的许多ufunc函数都是用C语言实现的，因此它们的运算速度非常快。

```
> x = np.linspace(0, 2*np.pi, 10)
> y = np.sin(x)
> y
array([ 0.00000000e+00,  6.42787610e-01,  9.84807753e-01,
        ...,
        -2.44929360e-16])
```

值得注意的是，对于同等长度的ndarray，`np.sin()`比`math.sin()`快，但是对于单个数值，`math.sin()`的速度则更快。

NumPy提供了许多ufunc函数，它们和相应的运算符运算结果相同。

```
> a = np.arange(0, 4)
> b = np.arange(1, 5)
> np.add(a, b)
array([1, 3, 5, 7])
```

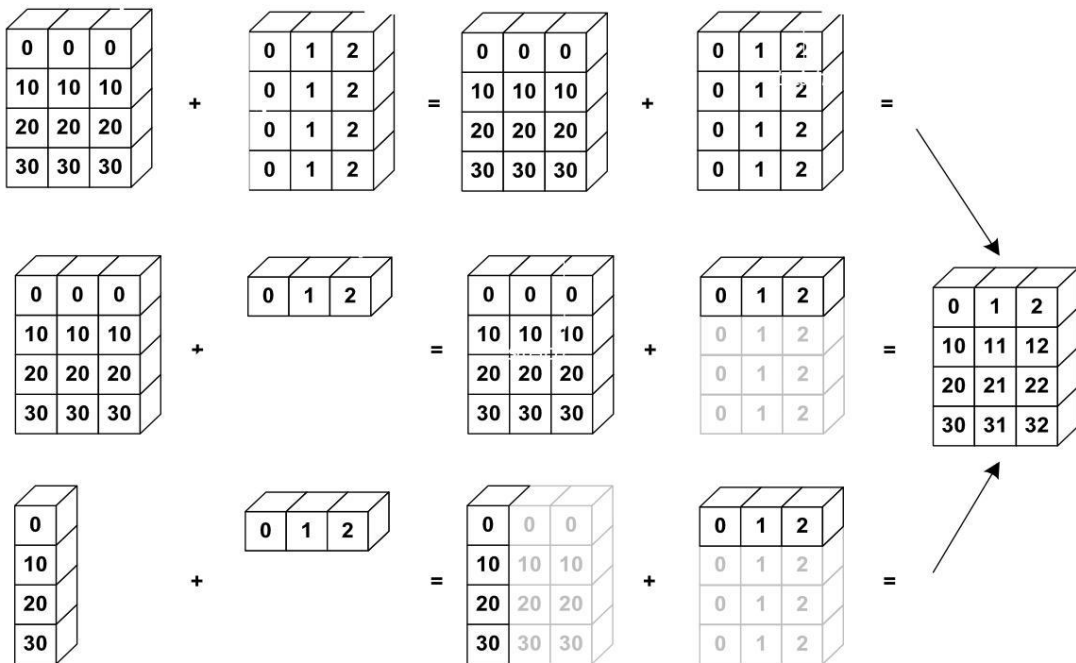
```
> a+b
array([1, 3, 5, 7])
```

```
> np.subtract(a, b) # 减法
> np.multiply(a, b) # 乘法
> np.divide(a, b) # 如果两个数字都为整数，则为整数除法
> np.power(a, b) # 乘方
```

运算	函数
$a + b$	<code>add(a,b)</code>
$a - b$	<code>subtract(a,b)</code>
$a * b$	<code>multiply(a,b)</code>
a / b	<code>divide(a,b)</code>
$a ** b$	<code>power(a,b)</code>
$a \% b$	<code>remainder(a,b)</code>

NumPy 核心优势是向量化运算，无需循环即可操作整个数组

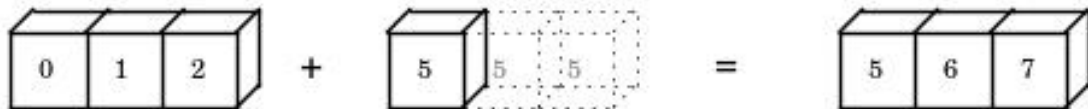
使用ufunc对两个数组进行运算时，ufunc函数会对两个数组的对应元素进行运算。
 如果数组的形状不相同，就会进行下广播处理。
 简而言之，就是向两个数组每一维度上的最大值靠齐。



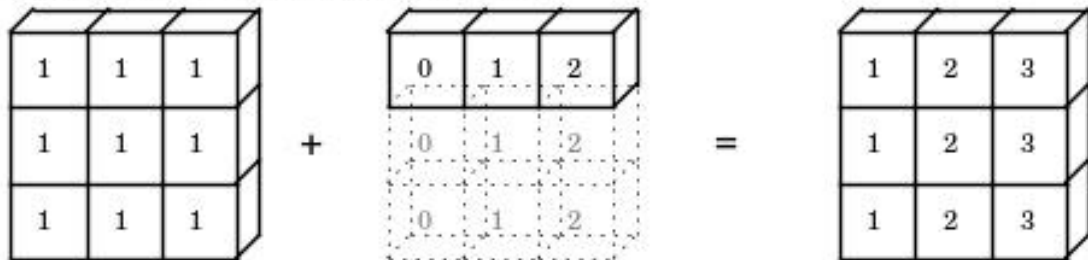
```
a = np.arange(0, 60, 10).reshape(-1, 1)
b = np.arange(0, 5)
c = a+b
c
```

```
----- c.shape -----
[[ 0, 1, 2, 3, 4],      (6, 5)
 [10, 11, 12, 13, 14],
 [20, 21, 22, 23, 24],
 [30, 31, 32, 33, 34],
 [40, 41, 42, 43, 44],
 [50, 51, 52, 53, 54]]
```

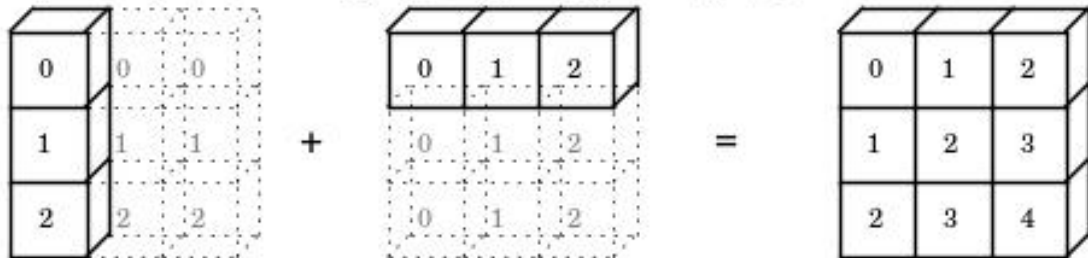
`np.arange(3) + 5`



`np.ones((3, 3)) + np.arange(3)`



`np.arange(3).reshape((3, 1)) + np.arange(3)`



矩阵运算 (线性代数)

```
mat1 = np.array([[1, 2], [3, 4]])
```

```
mat2 = np.array([[5, 6], [7, 8]])
```

```
print(np.dot(mat1, mat2)) # 矩阵乘法 (输出: [[19 22], [43 50]])
```

```
print(mat1 @ mat2) # 等价于np.dot() (Python 3.5+)
```

```
> np.random.seed(42)
```

```
> a = np.random.randint(0,10,size=(4,5))
```

```
> np.sum(a)
```

```
96
```

a	np.sum(a, axis=1)	np.sum(a, axis=0)
[[6, 3, 7, 4, 6],	[26, 28, 24, 18]	[22, 13, 27, 18, 16]
[9, 2, 6, 7, 4],		
[3, 7, 7, 2, 5],		
[4, 1, 7, 5, 1]]		

方法

作用

a.sum(axis=None)

求和

a.prod(axis=None)

求积

a.min(axis=None)

最小值

a.max(axis=None)

最大值

a.argmin(axis=None)

最小值索引

a.argmax(axis=None)

最大值索引

a.ptp(axis=None)

最大值减最小值

a.mean(axis=None)

平均值

a.std(axis=None)

标准差

a.var(axis=None)

方差

使用`==`, `>`对两个数组进行比较, 会返回一个布尔数组, 每一个元素都是对应元素的比较结果。

```
all_preds = np.array([3,2,1])  
all_labels = np.array([1,2,3])  
all_preds == all_labels
```

```
array([False, True, False])
```

```
test_accuracy = np.sum(all_preds == all_labels) / len(all_labels)  
print(test_accuracy)
```

```
0.3333333333333333
```

布尔运算在NumPy中也有对应的ufunc函数。

表达式	ufunc函数
$y=x1==x2$	<code>equal(x1,x2[,y])</code>
$y=x1!=x2$	<code>not_equal(x1,x2[,y])</code>
$y=x1<x2$	<code>less(x1,x2[,y])</code>
$y=x1<=x2$	<code>less_equal(x1,x2[,y])</code>
$y=x1>x2$	<code>greater(x1,x2[,y])</code>
$y=x1>=x2$	<code>greater_equal(x1,x2[,y])</code>

多维数组可以进行连接，分段等多种操作。我们先来看`vstack()`，`hstack()`，`column_stack()`函数。

```
> a = np.arange(3)
> b = np.arange(10, 13)

> v = np.vstack((a, b)) # 按第1轴连接数组
> h = np.hstack((a, b)) # 按第0轴连接数组
> c = np.column_stack((a, b)) # 按列连接多个一维数组
```

v	h	c
-----	-----	-----
[[0, 1, 2], [10, 11, 12]]	[0, 1, 2, 10, 11, 12]	[[0, 10], [1, 11], [2, 12]]

split()函数进行分段。

```
a = np.array([6, 3, 7, 4, 6, 9, 2, 6, 7, 4, 3, 7])  
b = np.array([ 1, 3, 6, 9, 10])  
np.split(a, b) # 按元素位置进行分段
```

```
[array([6]),  
array([3, 7]),  
array([4, 6, 9]),  
array([2, 6, 7]),  
array([4]),  
array([3, 7])]
```

```
np.split(a, 2) # 按数组个数进行分段
```

```
[array([6, 3, 7, 4, 6, 9]),  
array([2, 6, 7, 4, 3, 7])]
```

savetxt 可以将数组写入文件，默认使用科学计数法的形式保存

```
import numpy as np
data = np.array([[1, 2],
                 [3, 4]])

# 保存文件
np.savetxt('out.txt', data)

# 读取文件
with open('out.txt') as f:
    for line in f:
        print(line)
```

```
# 读取文件
np.loadtxt('out.txt')
```

```
array([[1., 2.],
       [3., 4.]])
```

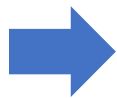
```
dataset = np.loadtxt(r'iris.data', dtype=str,
                    delimiter=',')
print(dataset)
x = dataset[:, :4].astype(float)
print(x)
```

1.000000000000000000e+00 2.000000000000000000e+00

3.000000000000000000e+00 4.000000000000000000e+00

AI库及使用

学习内容 CONTENTS



1

Numpy

2

Pandas

3

Matplotlib

4

Scikit-learn

Pandas 是 Python 中最常用到的数据操作和分析工具包，它构建在 Numpy 之上，主要基于其 Dataframe 对象处理表格型或异质型数据，而 Numpy（提供高性能的矩阵运算）更适合处理同质的数值类型数据。

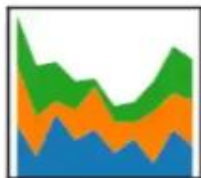
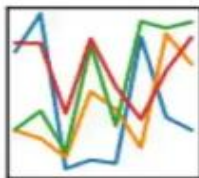
Pandas库的功能:

- 可用于数据挖掘和数据分析，同时也提供数据清洗功能；
- 支持类似SQL的数据增、删、查、改，并且带有丰富的数据处理函数；
- 支持时间序列分析功能；支持灵活处理缺失数据等。



pandas

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$



● 基本数据结构

Series

一维数据，一种类似于一维数组的对象，包含行索引和数据两个部分

索引 s.index	数据 s.values
张某	22
李某	26
段某	24

DataFrame

二维数据，表格类型的数据结构，包含带索引的多列数据，各列的数据类型（数值、字符串、布尔型等）可能不同

列索引名称 df.columns.names	个人信息	年龄	籍贯	职业	列索引 (列名) df.columns
行索引名称 df.index.names	姓名				
行索引 df.index	张某	22	北京	律师	数据 df.values
	李某	26	四川成都	工程师	
	段某	24	江苏南京	研究员	

```
import pandas as pd
```

```
# 创建 Series: 索引+数据
```

```
s = pd.Series(  
    data=[22, 26, 24],  
    index=["张某", "李某", "段某"],  
    name="年龄"  
)
```

```
#官方构造函数  
pandas.Series(  
    data=None,          # 数据源  
    index=None,         # 索引  
    dtype=None,         # 数据类型  
    name=None,          # Series名称  
    copy=False,         # 是否复制数据  
    fastpath=False      # 内部优化参数, 用户一般不用  
)
```

张某	22
李某	26
段某	24

索引 — s.index 数据 — s.values

```
# 核心属性 (官方关注)
```

```
print("值: ", s.values)
```

```
print("索引: ", s.index)
```

```
print("数据类型: ", s.dtype)
```

```
print("名称: ", s.name)
```

```
# 底层 NumPy 数组: [22 26 24]
```

```
# 标签索引: Index(['张某', '李某', '段某'], dtype='object')
```

```
# int64
```

```
# 年龄
```

```
# 创建 DataFrame: 数据+行索引+列索引
df = pd.DataFrame(
    data={
        "年龄": [22, 26, 24],
        "籍贯": ["北京", "四川成都", "江苏南京"],
        "职业": ["律师", "工程师", "研究员"]
    },
    index=["张某", "李某", "段某"]
)
```

```
# 核心属性
print("行索引: ", df.index)
print("列索引: ", df.columns)
print("数据类型 (列级): ", df.dtypes)
print("数据值: ", df.values)      # 二维 NumPy 数组
print(df.head())                 # head 方法可查看最前面几行数据 (默认为 5)
print(df.tail(1))                # tail 方法可查看最后面几行的数据 (默认为 5)
```

```
# 官方构造函数
pandas.DataFrame(
    data=None,           # 数据源
    index=None,          # 行索引
    columns=None,        # 列索引
    dtype=None,          # 数据类型
    copy=None            # 是否复制数据
)
```

列索引名称
df.columns.names

行索引名称
df.index.names

行索引
df.index

列索引 (列名)
df.columns

数据
df.values

个人信息	年龄	籍贯	职业
姓名			
张某	22	北京	律师
李某	26	四川成都	工程师
段某	24	江苏南京	研究员

文件读写

从文件中读取数据 (DataFrame)

`pd.read_csv()`: 从CSV文件读取.

`pd.read_table()`: 从制表符分隔文件读取, 如TSV.

`pd.read_excel()`: 从 Excel 文件读取.

`pd.read_sql()`: 从 SQL 表或数据库读取.

`pd.read_json()`: 从JSON格式的URL或文件读取.

`pd.read_clipboard()`: 从剪切板读取.

将DataFrame写入文件

`df.to_csv()`: 写入CSV文件.

`df.to_excel()`: 写入Excel文件.

`df.to_sql()`: 写入SQL表或数据库.

`df.to_json()`: 写入JSON格式的文件.

`df.to_clipboard()`: 写入剪切板.

```
import pandas as pd
df = pd.DataFrame(
    data={
        "年龄": [22, 26, 24],
        "籍贯": ["北京", "四川成都", "江苏南京"],
        "职业": ["律师", "工程师", "研究员"]
    },
    index=["张某", "李某", "段某"]
)
df.to_csv('test.csv') #写文件

data_path = r'.\iris.txt' # 加载txt文件
iris = pd.read_csv(data_path, delimiter=" ")
print(iris.head())
```

- 数据索引,推荐使用 .loc, .iloc方法进行索引

#1. 访问行

`df[1:3]` #索引从 1 到 2 的行
通过切片方式选取多行

#2. 访问列

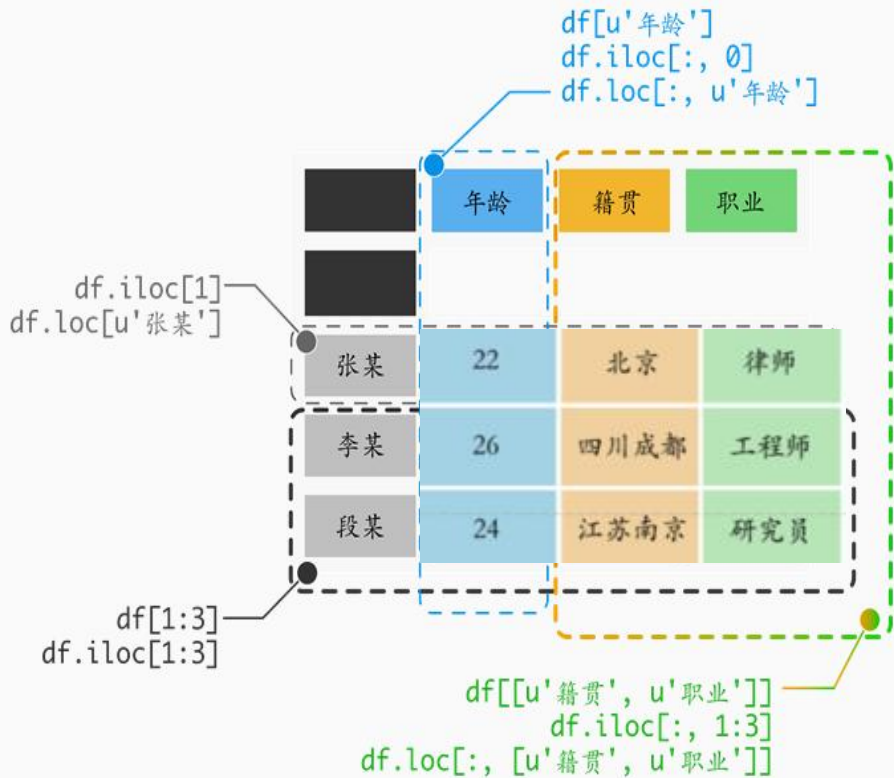
`df[col_label]` 或 `df.col_label`
选取某列, col_label 为列的标签 (列名)

#3. 基于标签 (行 / 列名) 访问

`df.loc[行标签,列标签]`
通过标签选取行/列, 支持单行/列, 多行/列

#4. 基于位置 (数字索引) 访问

`df.iloc[行位置索引, 列位置索引]`
通过位置 (自然数) 选取行/列



- 数据索引,推荐使用 .loc, .iloc方法进行索引

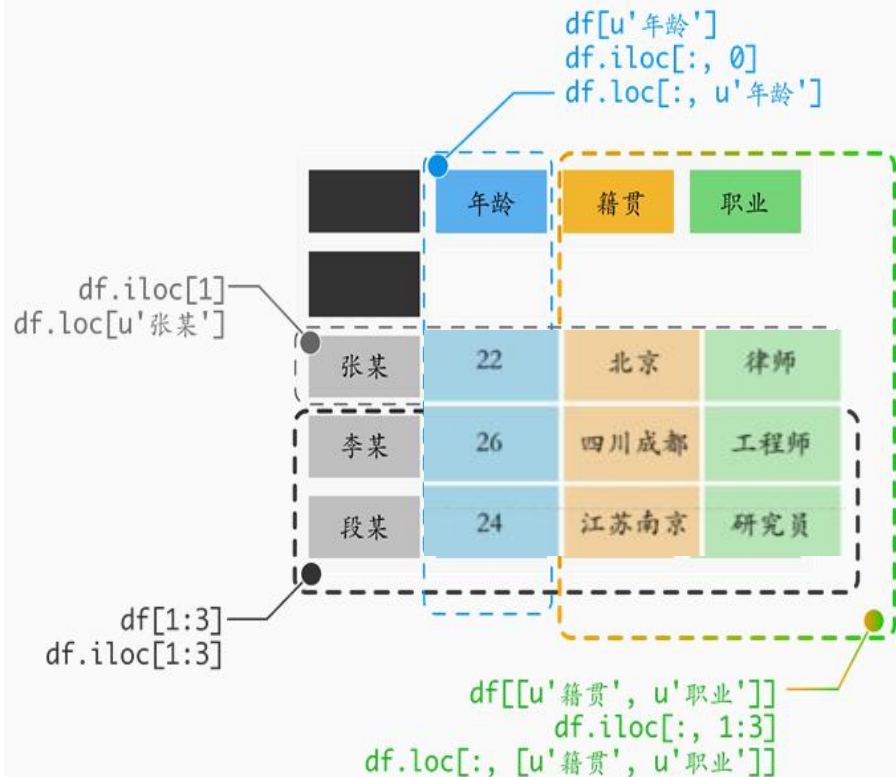
#5.布尔型索引访问

```
df[df.年龄 > 25]
```

	年龄	籍贯	职业
李某	26	四川成都	工程师

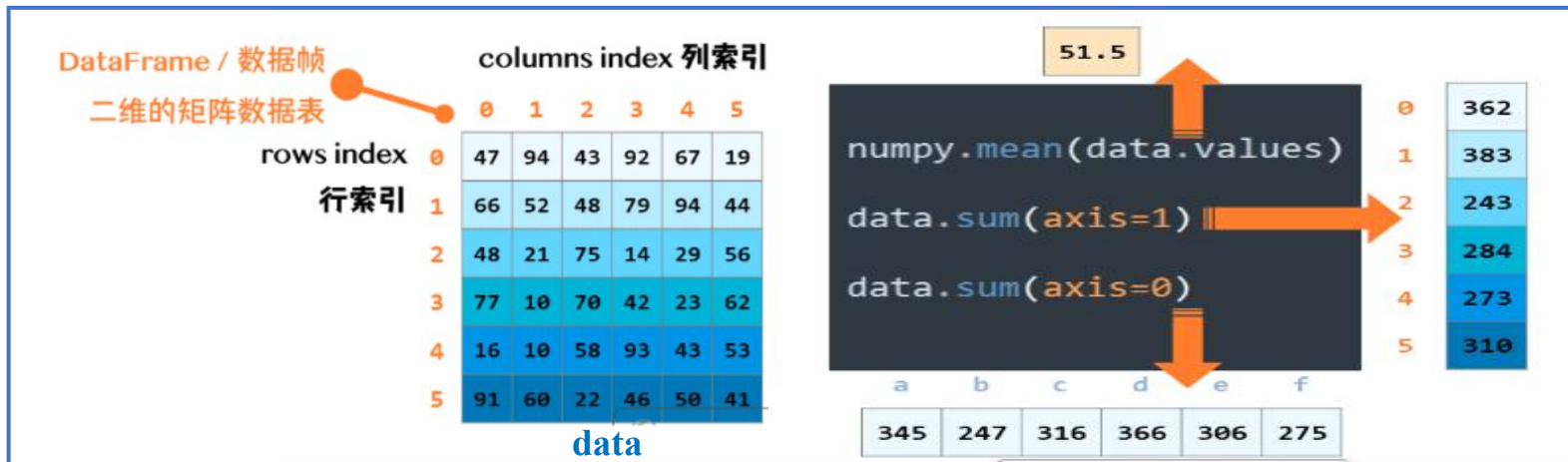
```
df[df['年龄'] > 25]
```

	年龄	籍贯	职业
李某	26	四川成都	工程师



Dataframe: 聚合

可以按行、列进行聚合, 也可以用 Pandas 内置的describe对数据进行操作简单而又全面的数据聚合分析。



1. `data.function(axis=0)` # 按列计算

2. `data.function(axis=1)` # 按行计算

Dataframe: 聚合函数



count	非NA值的计数	var	方差
describe	计算汇总统计	std	标准差
min、max	计算最小值、最大值	skew	偏度
argmin、argmax	计算能够获取的最小/大值的索引位置	kurt	峰度
idmin、idmax	计算能够获取的最小/大值的索引值	cumsum	累计求和
quantile	计算样本的分位数	cummin、cummax	累计最小/大值
sum	求和	cumprod	累计积
mean	mean均值	diff	差分
median	中位数(同 quantile(0.58))	mad	平均绝对偏差

Dataframe: 分组统计

可以按照指定的多列进行指定的多个运算进行汇总统计。

	a	b	c	d	e	f	g	h
0	47	94	43	92	67	19	GD	GZ
1	66	52	48	79	94	44	GD	SZ
2	48	21	75	14	29	56	FJ	FZ
3	77	10	70	42	23	62	FJ	FZ
4	16	10	58	93	43	53	GX	NN
5	91	60	22	46	50	41	GX	NN

g	a	b	c	d	e	f
FJ	125	31	145	56	52	118
GD	113	146	91	171	161	63
GX	107	70	80	139	93	94

`df.groupby('g').sum()`

g	sum	mean	std
FJ	56	28	20
GD	171	86	9.2
GX	139	70	33

`df.groupby('g')['d'].agg([numpy.sum, numpy.mean, numpy.std])`

g	h	a	b	c	d	e	f
FJ	FZ	63	16	73	28	26	59
GD	GZ	47	94	43	92	67	19
	SZ	66	52	48	79	94	44
GX	NN	54	35	40	70	47	47

`df.groupby(['g', 'h']).mean()`

Dataframe处理缺失值

删除含有缺失值的行

	a	b	c	d	e	f	g
0	47	94	43	92	67	19	GD
1	66	52	NaN	79	94	44	GX
2	48	21	75	14	29	56	BJ
3	77	10	70	42	NaN	62	SH
4	16	10	58	93	43	53	HN
5	91	60	22	46	50	41	TJ

```
data.dropna(axis=1)
```

	a	b	c	d	e	f	g
0	47	94	43	92	67	19	GD
1	66	52	NaN	79	94	44	GX
2	48	21	75	14	29	56	BJ
3	77	10	70	42	NaN	62	SH
4	16	10	58	93	43	53	HN
5	91	60	22	46	50	41	TJ

删除含有缺失值的列

	a	b	c	d	e	f	g
0	47	94	43	92	67	19	GD
1	66	52	NaN	79	94	44	GX
2	48	21	75	14	29	56	BJ
3	77	10	70	42	NaN	62	SH
4	16	10	58	93	43	53	HN
5	91	60	22	46	50	41	TJ

```
data.dropna(axis=0)
```

	a	b	c	d	e	f	g
0	47	94	43	92	67	19	GD
1	66	52	999 53.6	79	94	44	GX
2	48	21	75	14	29	56	BJ
3	77	10	70	42	999 56.6	62	SH
4	16	10	58	93	43	53	HN
5	91	60	22	46	50	41	TJ

```
data.fillna(999)
```

```
data.fillna(data.mean(numeric_only=True))
```



#检查缺失数据的位置

```
pd.isnull(data)
```

[illegible]

用填充的办法

```
import pandas as pd
data = pd.read_csv('chip_test.csv')
data.head()
```

```
x = data.drop(['pass'], axis = 1)
# axis = 1表示按列删除
x
```

```
y = data.loc[:, 'pass']
y
```

```
pd.value_counts(y)
```

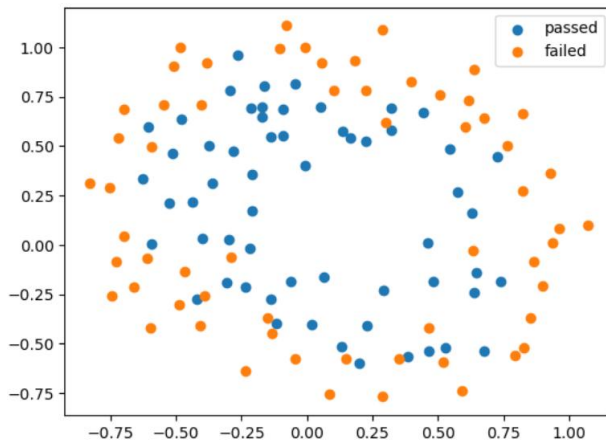
```
pass
0    60
1    58
Name: count, dtype: int64
```

	test1	test2
0	0.051267	0.699560
1	-0.092742	0.684940
2	-0.213710	0.692250
3	-0.375000	0.502190
4	0.183760	0.933480
...	...	...
113	-0.720620	0.538740
114	-0.593890	0.494880
115	-0.484450	0.999270
116	-0.006336	0.999270
117	0.632650	-0.030612

118 rows × 2 columns

```
0    1
1    1
2    1
3    1
4    0
..
113  0
114  0
115  0
116  0
117  0
Name: pass, Length: 118, dtype: int64
```

	test1	test2	pass
0	0.051267	0.69956	1
1	-0.092742	0.68494	1
2	-0.213710	0.69225	1
3	-0.375000	0.50219	1
4	0.183760	0.93348	0



● 数据合并

`pd.merge(left, right)`, 会自动按照索引对齐, 可指定两个 DataFrame 的对齐方式

`how`: 合并方式, 包括左连接、右连接、内连接 (默认) 和外连接

`on`: 连接键, 用于合并的列名

`left_on`: 左键; `right_on`: 右键

`left_index`: 是否将左 DataFrame 行索引作为左键

`right_index`: 是否将右 DataFrame 行索引作为右键

	姓名	年龄
0	张某	22
1	李某	26
2	段某	24

	姓名	籍贯
7	张某	北京
8	李某	四川成都
9	钱某	江苏南京

inner

	姓名	年龄	籍贯
0	张某	22	北京
1	李某	26	四川成都

```
pd.merge(left, right,
         how='inner', on='姓名')
```

outer

	姓名	年龄	籍贯
0	张某	22.0	北京
1	李某	26.0	四川成都
2	段某	24.0	NaN
3	钱某	NaN	江苏南京

```
pd.merge(left, right,
         how='outer', on='姓名')
```

left

	姓名	年龄	籍贯
0	张某	22	北京
1	李某	26	四川成都
2	段某	24	NaN

```
pd.merge(left, right,
         how='left', on='姓名')
```

right

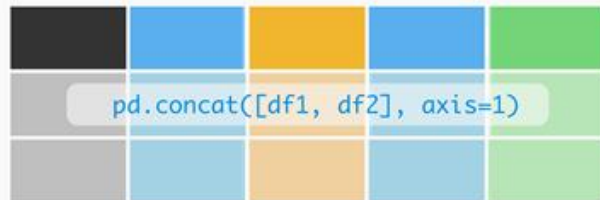
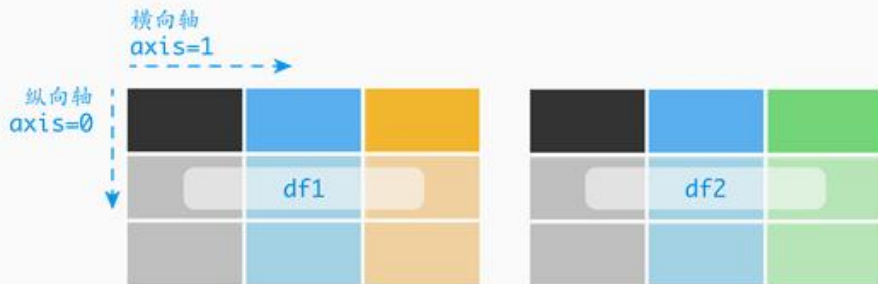
	姓名	年龄	籍贯
0	张某	22.0	北京
1	李某	26.0	四川成都
2	钱某	NaN	江苏南京

```
pd.merge(left, right,
         how='right', on='姓名')
```

- 数据融合

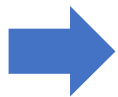
`pd.concat([df1, df2])`

轴向连接多个DataFrame



AI库及使用

学习内容 CONTENTS



1

Numpy

2

Pandas

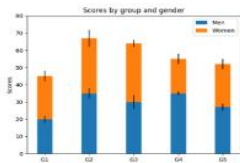
3

Matplotlib

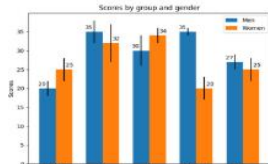
4

Scikit-learn

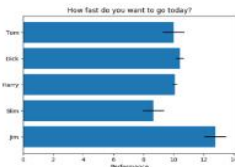
Lines, bars and markers



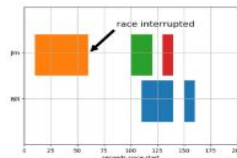
Stacked Bar Graph



Grouped bar chart with labels



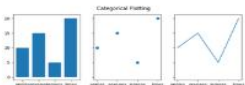
Horizontal bar chart



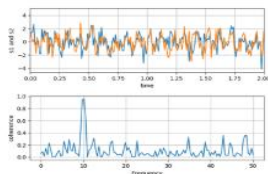
Broken Barh

➤ Matplotlib 是一个 Python 的 2D 绘图库。

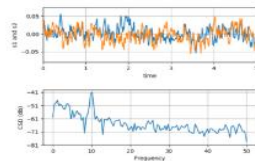
➤ 通过 Matplotlib, 可以生成绘图, 直方图, 功率谱, 条形图, 错误图, 散点图等。



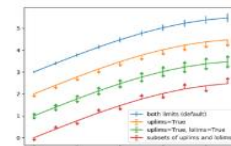
Plotting categorical variables



Plotting the coherence of two signals

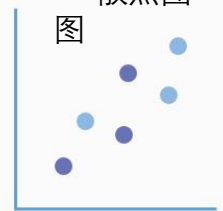


CSD Demo



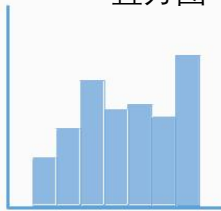
Errorbar limit selection

散点图



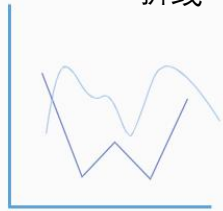
plt.scatter()

直方图



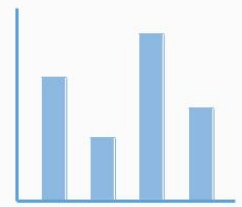
plt.hist()

折线



plt.plot()

柱状图



plt.bar()

水平条形图

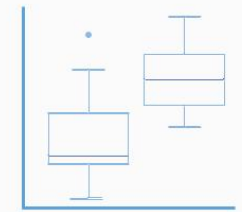


plt.barh()

饼图

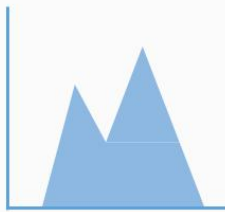


plt.pie()



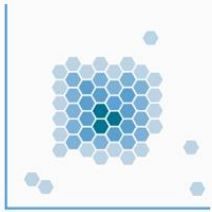
plt.boxplot()

箱线图



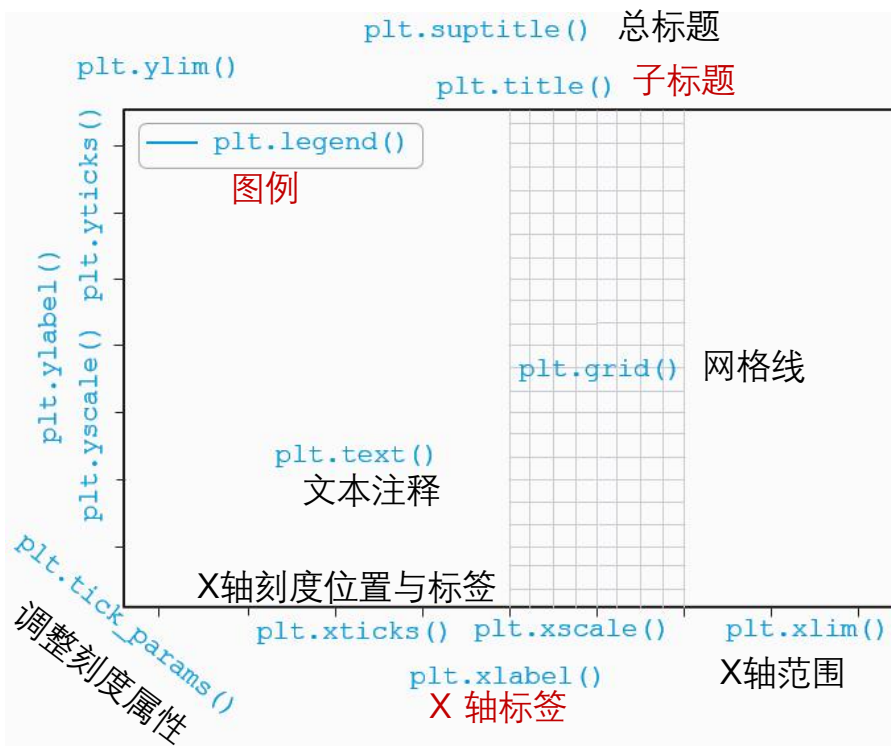
plt.fill()

填充图



plt.hexbin()

六边形密度分布



图形标注与修饰

```
import matplotlib.pyplot as plt
import numpy as np
# 简单的绘图
x = np.linspace(0, 2 * np.pi, 50)
```

如果没有第一个参数 x, 图形的 x 坐标默认为数组的索引
`plt.plot(x, np.sin(x))`

显示图形
`plt.show()`

```
plt.plot(x, np.sin(x), 'r-^',
         x, np.cos(x), 'g--')
plt.show()
```

• 颜色:

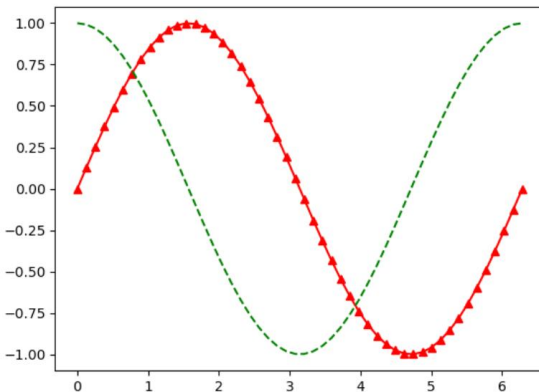
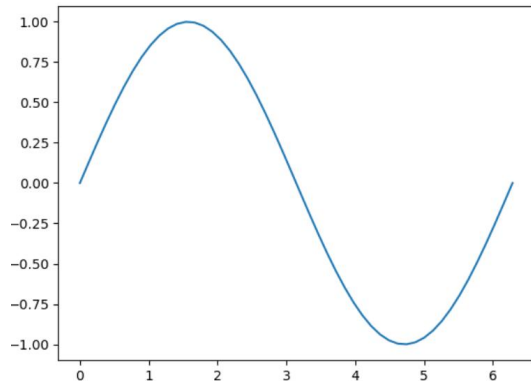
- 蓝色 - 'b'
- 绿色 - 'g'
- 红色 - 'r'
- 青色 - 'c'
- 品红 - 'm'
- 黄色 - 'y'
- 黑色 - 'k' ('b'代表蓝色, 所以这里用黑色的最后一个字母)
- 白色 - 'w'

• 线:

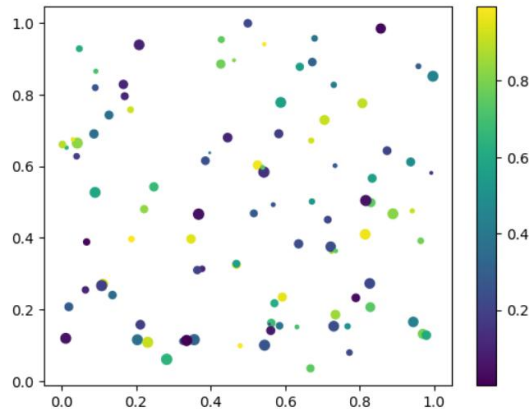
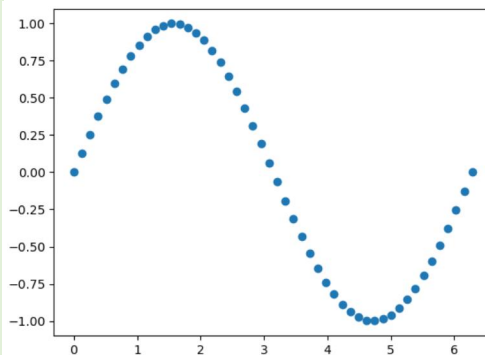
- 直线 - '-'
- 虚线 - '--'
- 点线 - '.'
- 点划线 - '-.'

• 常用点标记:

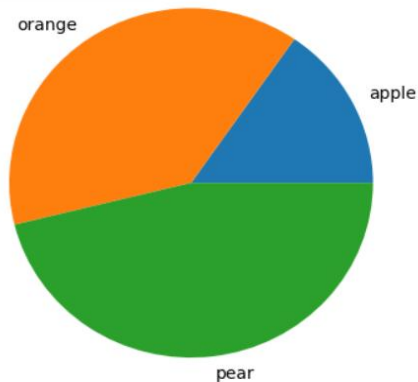
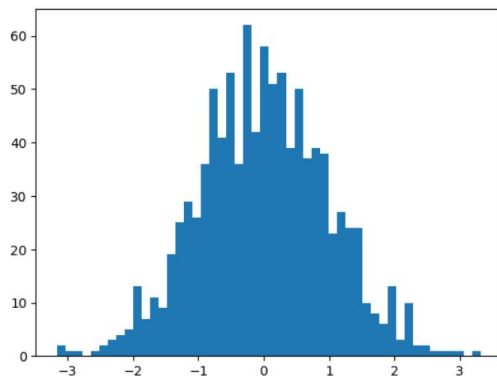
- 点 - '.'
- 像素 - ','
- 圆 - 'o'
- 方形 - 's'
- 三角形 - '^'



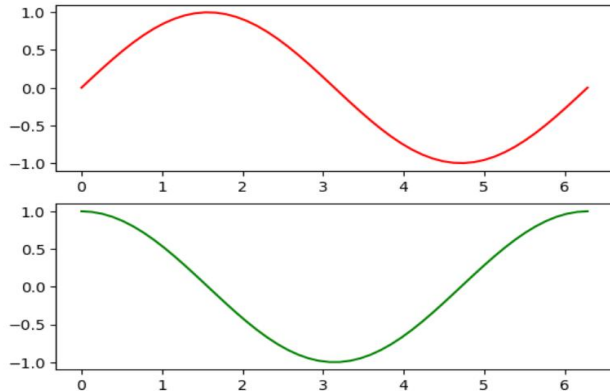
```
plt.scatter(x,np.sin(x)) # 散点图
plt.show()
x = np.random.rand(100)
y = np.random.rand(100)
size = np.random.rand(100) * 50
color = np.random.rand(100)
plt.scatter(x, y, size, color) # 散点图
plt.colorbar()
plt.show()
```



```
x = np.random.randn(1000)
plt.hist(x, 50) # 直方图
plt.show()
fruits = ['apple', 'orange', 'pear']
sales = [100, 250, 300]
plt.pie(sales, labels=fruits) # 饼图
plt.savefig('pie.png')
plt.show()
```

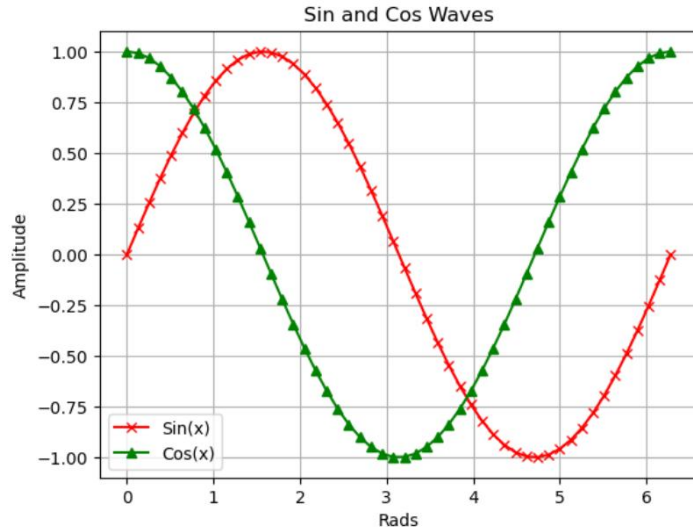


```
x = np.linspace(0, 2 * np.pi, 50)
plt.subplot(2, 1, 1) # (行, 列, 活跃区)
plt.plot(x, np.sin(x), 'r')
plt.subplot(2, 1, 2)
plt.plot(x, np.cos(x), 'g')
plt.show()
```



```
x = np.linspace(0, 2 * np.pi, 50)
plt.plot(x, np.sin(x), 'r-x', label='Sin(x)')
plt.plot(x, np.cos(x), 'g-^', label='Cos(x)')
```

```
plt.legend() # 展示图例
plt.xlabel('Rads') # 给 x 轴添加标签
plt.ylabel('Amplitude') # 给 y 轴添加标签
plt.title('Sin and Cos Waves') # 添加图形标题
plt.grid() # 添加网格线
plt.show()
```



```
import matplotlib.pyplot as plt
import numpy as np
# 模拟数据: 一天中每2小时的温度 (0-22点)
x = np.arange(0, 24, 2)
y = np.array([14, 9, 7, 5, 12, 19, 23, 26, 27, 24, 21, 19])
```

1. 创建画布与子图

```
plt.figure(figsize=(10, 6))
```

2. 绘图 + 图例

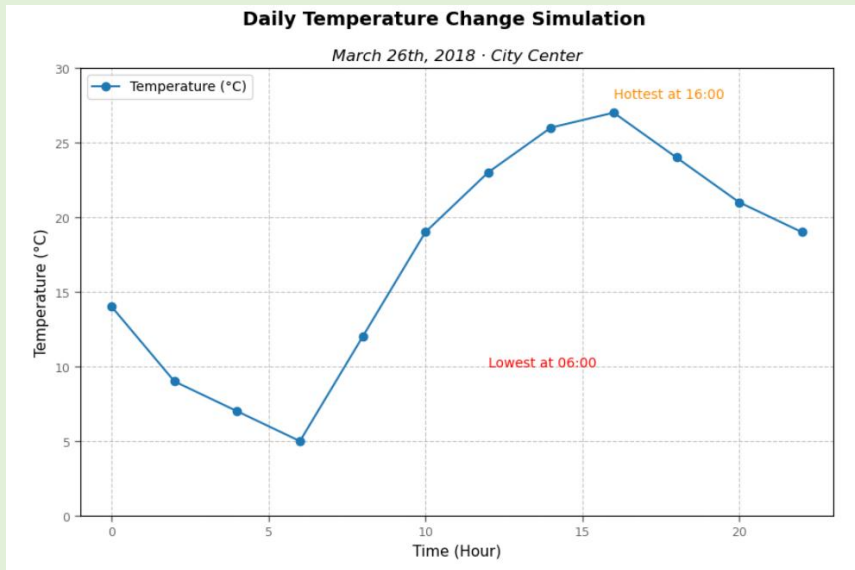
```
plt.plot(x, y, '-o', color='#1f77b4', label='Temperature ( ° C)')
plt.legend(loc='upper left', frameon=True)
```

3. 标题与子标题

```
plt.suptitle('Daily Temperature Change Simulation', fontsize=14, weight='bold', y=0.98)
plt.title('March 26th, 2018 · City Center', fontsize=12, style='italic')
```

4. 坐标轴标签

```
plt.xlabel('Time (Hour)', fontsize=11)
plt.ylabel('Temperature ( ° C)', fontsize=11)
```



5. 坐标轴刻度与范围

```
plt.xticks(x, labels=[f'{h}:00' for h in x]) # 自定义x轴刻度标签
```

```
plt.yticks(np.arange(0, 30, 5)) # 自定义y轴刻度
```

```
plt.xlim(-1, 23) # x轴范围
```

```
plt.ylim(0, 30) # y轴范围
```

6. 坐标轴缩放

```
plt.xscale('linear')
```

```
plt.yscale('linear') # 若想用对数, 可改为 'log'
```

```
plt.grid(True, linestyle='--', alpha=0.7) # 7. 网格
```

8. 文本标注

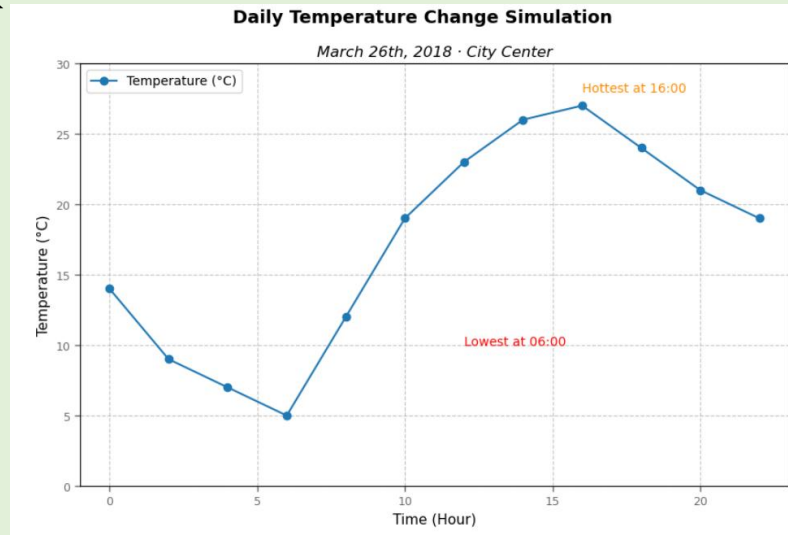
```
plt.text(12, 10, 'Lowest at 06:00', fontsize=10, color='red')
```

```
plt.text(16, 28, 'Hottest at 16:00', fontsize=10, color='darkorange')
```

9. 刻度样式调整

```
plt.tick_params(axis='both', labelsize=9, colors='dimgray', length=4, width=1)
```

```
plt.show() # 10. 显示图形
```

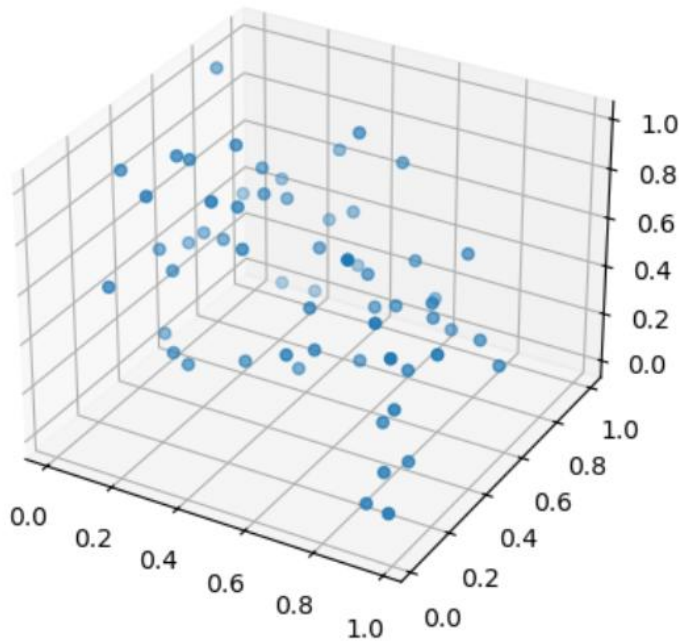


```
import matplotlib.pyplot as plt
import numpy as np

# 一步创建3D绘图对象
fig, axe = plt.subplots(subplot_kw={'projection': '3d'})

# 随机数据生成
np.random.seed(100)
# 一次性生成3行60列随机数, 解包为x/y/z
x, y, z = np.random.rand(3, 60)

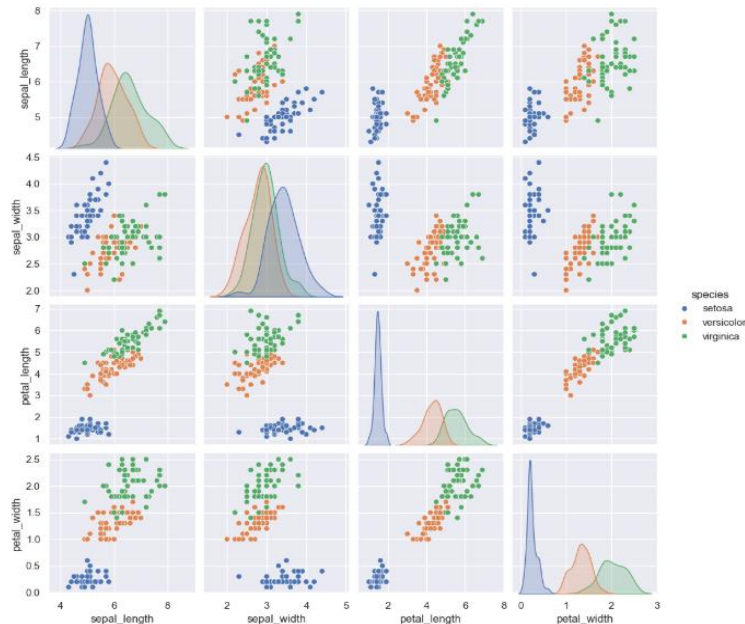
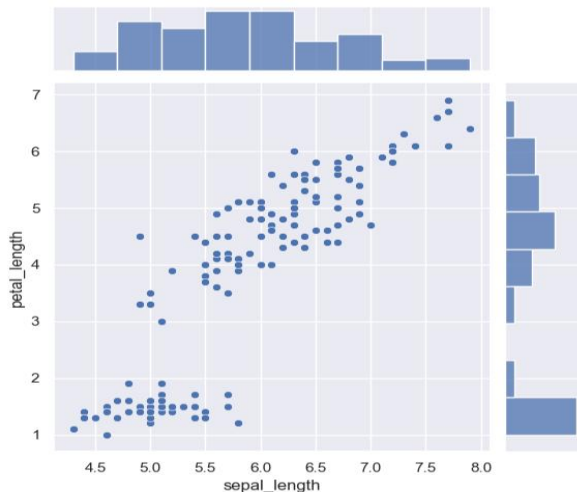
# 绘图+显示
axe.scatter(x, y, z)
plt.show()
```



Seaborn 基于 matplotlib， 可以快速的绘制一些统计图表。

```
import seaborn as sns
import pandas as pd
sns.set()
iris = pd.read_csv("iris.csv")
sns.jointplot(x="sepal_length", y="petal_length", data=iris)

sns.pairplot(data=iris, hue="species")
```



AI库及使用

学习内容 CONTENTS

1

Numpy

2

Pandas

3

Matplotlib

4

Scikit-learn



📦 Scikit-learn简称 sklearn，是基于NumPy、SciPy和 Matplotlib的开源Python机器学习包，封装了一系列数据预处理、机器学习算法、模型选择等工具。

📦 自2007年发布以来，scikit-learn已经成为Python重要的机器学习库了，支持包括分类，回归，降维和聚类四大机器学习算法，还包括了特征提取，数据处理和模型评估三大模块。

导入工具包：

```
from sklearn import datasets, preprocessing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
```



SKLearn 官网: <https://scikit-learn.org/stable/>

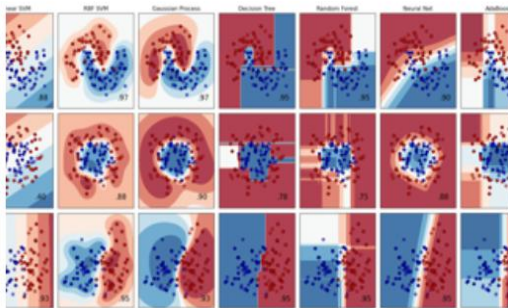
分类

Classification

Identifying which category an object belongs to.

Applications: Spam detection, image recognition.

Algorithms: [Gradient boosting](#), [nearest neighbors](#), [random forest](#), [logistic regression](#), and [more...](#)



Examples

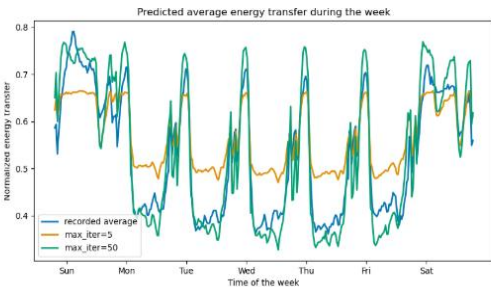
回归

Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, stock prices.

Algorithms: [Gradient boosting](#), [nearest neighbors](#), [random forest](#), [ridge](#), and [more...](#)



Examples

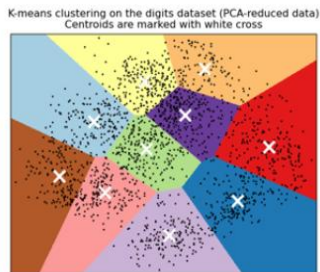
聚类

Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, grouping experiment outcomes.

Algorithms: [k-Means](#), [HDBSCAN](#), [hierarchical clustering](#), and [more...](#)



Examples

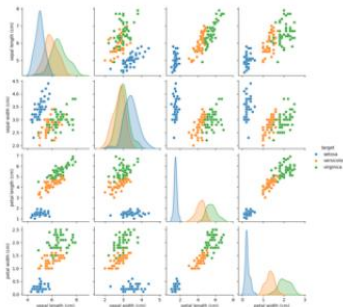
降维

Dimensionality reduction

Reducing the number of random variables to consider.

Applications: Visualization, increased efficiency.

Algorithms: [PCA](#), [feature selection](#), [non-negative matrix factorization](#), and [more...](#)



Examples

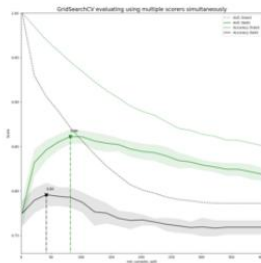
模型选择

Model selection

Comparing, validating and choosing parameters and models.

Applications: Improved accuracy via parameter tuning.

Algorithms: [Grid search](#), [cross validation](#), [metrics](#), and [more...](#)



Examples

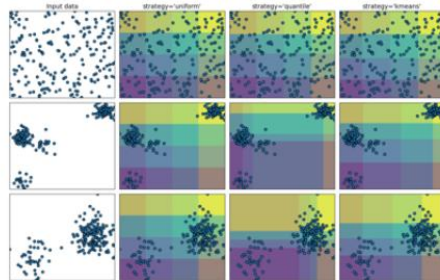
数据预处理

Preprocessing

Feature extraction and normalization.

Applications: Transforming input data such as text for use with machine learning algorithms.

Algorithms: [Preprocessing](#), [feature extraction](#), and [more...](#)



Examples